

文献分享：Agentic Context Engineering

0 论文信息

1 研究背景

1.1 技术背景

1.1.1 上下文工程 (Context Engineering)

1.1.2 离线/在线适应 (Offline Adaptation, Online Adaptation)

1.1.3 上下文适应 VS 权重适应

1.1.4 长上下文LLM与KV缓存复用

1.1.5 智能体与ReAct架构

1.2 问题背景

1.2.1 简洁性偏差

1.2.2 上下文坍塌

1.3 相关工作

1.3.1 ICL

1.3.2 GEPA

1.3.3 Dynamic Cheatsheet

2 模型与方法

2.1 ACE框架

2.1.1 生成器 (Generator)

2.1.2 反思器 (Reflector)

2.1.3 整理器 (Curator)

2.2 实现细节

2.2.1 ContextPlaybook

2.2.2 增量式差异更新

2.2.3 冗余控制

2.2.4 离线预热

3 数据与实验

3.1 基准实验

3.1.1 AppWorld Agent Benchmark

3.1.2 Financial Analysis Benchmark

3.2 消融实验

3.3 性能与成本对比

4 总结与展望

4.1 创新点

4.2 局限性

4.3 成果公开

个人思考

0 论文信息

题目：✗ [Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models](#)

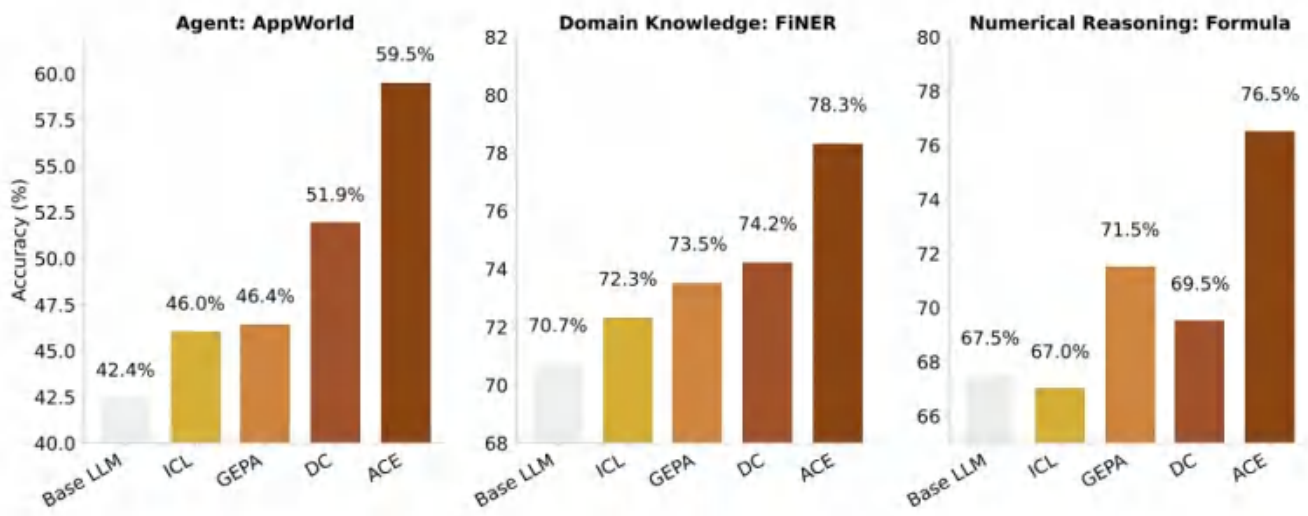
作者：Qizheng Zhang; Changran Hu 等

机构：Stanford University

概括：提出 ACE (Agentic Context Engineering, 智能体上下文工程) 框架，动态管理上下文，渐进式积累知识和经验，优化大语言模型在智能体任务和专业领域任务的表现。

表现：

两个领域，三个基准，SOTA



1 研究背景

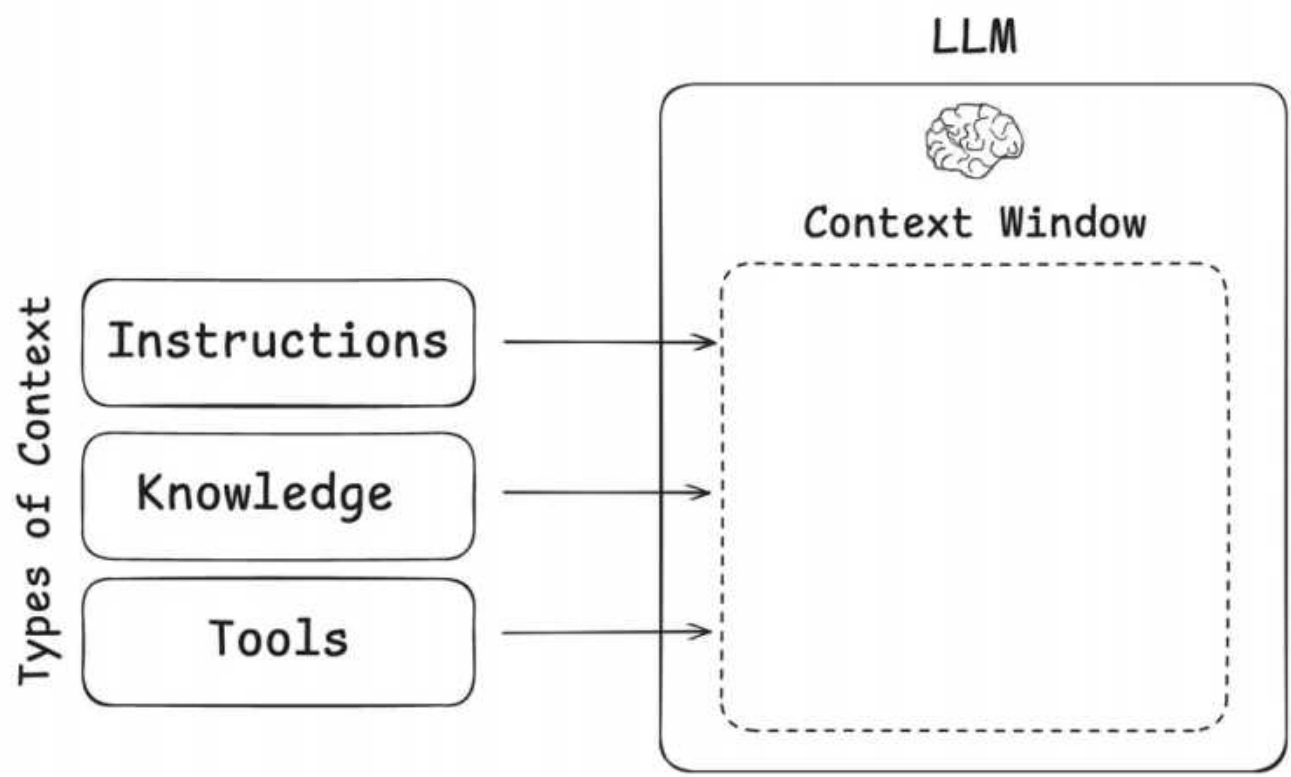
1.1 技术背景

1.1.1 上下文工程（Context Engineering）

随着AI 智能体的兴起，很多失败并非源于模型本身，而是源于上下文信息不足或不准确，上下文工程正是解决这一问题的核心技术。

上下文工程是构建系统，通过动态地提供给大型语言模型（LLM）正确格式的信息和工具，使其能够可靠地完成任务。它是一个比“提示工程”更广泛的概念，涵盖了信息源管理、上下文构建逻辑、动态调度机制和工具整合等多个方面。核心目标是通过管理和优化模型获取的“上下文”，提高模型的性能和可靠性。

上下文工程是提示词工程的超类，包含了信息收集和系统构建的整个过程。



1.1.2 离线/在线适应（Offline Adaptation, Online Adaptation）

离线适应指在模型部署到实际任务（如智能体交互、金融推理）前，利用训练数据集或预先收集的信息，对上下文（如系统提示、基础策略库）进行优化的过程。其核心目标是构建一个**初始的**、高质量的上下文模板，供模型在实际执行任务时直接调用，无需实时调整。

在线适应指在模型**实际执行任务的过程中**（如智能体与环境实时交互、处理用户动态查询时），基于实时产生的反馈信号（如执行结果、错误信息），**动态更新上下文**的过程。其核心目标是让上下文随任务推进持续进化，弥补离线适应无法覆盖的“动态场景”（如未知错误、新任务模式）。

1.1.3 上下文适应 VS 权重适应

智能系统的改进策略：

上下文适应	权重适应
上下文是自然语言，具有 可解释性	模型权重是数字，难以直观分析和解释
能够 运行时 快速整理 新知识	需要通过模型训练更新知识
可在多个模型或模块之间 信息共享	难以进行模型间交流

1.1.4 长上下文LLM与KV缓存复用

基于相对位置编码技术，LLM的上下文长度**理论上无限长**。

上下文窗口大小的**瓶颈是计算资源**：显存占用关于序列长度的空间复杂度为 $O(n^2)$ 。

随着计算优化技术的发展，LLM支持的上下文越来越长：

qwen-turbo支持8k上下文：

模型限流与上下文			
上下文长度	8K	最大输出长度	2K
最大输入长度	6K	TPM	60000
RPM	60		

deepseek-R1支持128K上下文：

模型限流与上下文			
上下文长度	128K	最大输出长度	64K
最大输入长度	96K	TPM	1200000
RPM	15000		

qwen-plus支持1M上下文（相当于15本《哈利波特》系列小说的总字数）：

模型限流与上下文			
上下文长度	1M	最大输出长度	32K
最大输入长度	995K	TPM	1200000
RPM	15000		

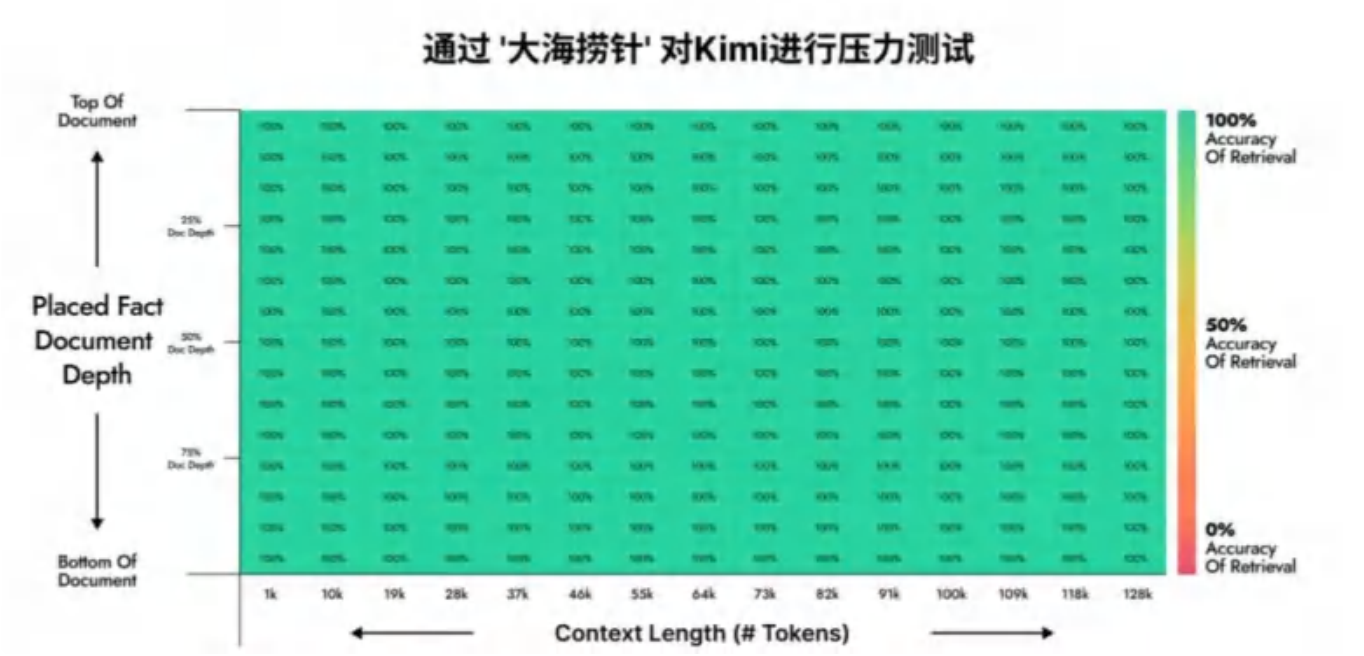
基于KV缓存技术，当用户查询命中缓存时，推理成本极大降低：

qwen-max

模型价格	阶梯计费	输入<=32k	切换百万tokens
输入	0.006	输出	0.024
	元/千Token		元/千Token
输入（缓存命中）	0.0012		
	元/千Token		

不同于人类短时记忆的局限，LLM在长上下文中捕获信息的能力很强：

评估方法——大海捞针实验（Needle In A Haystack）

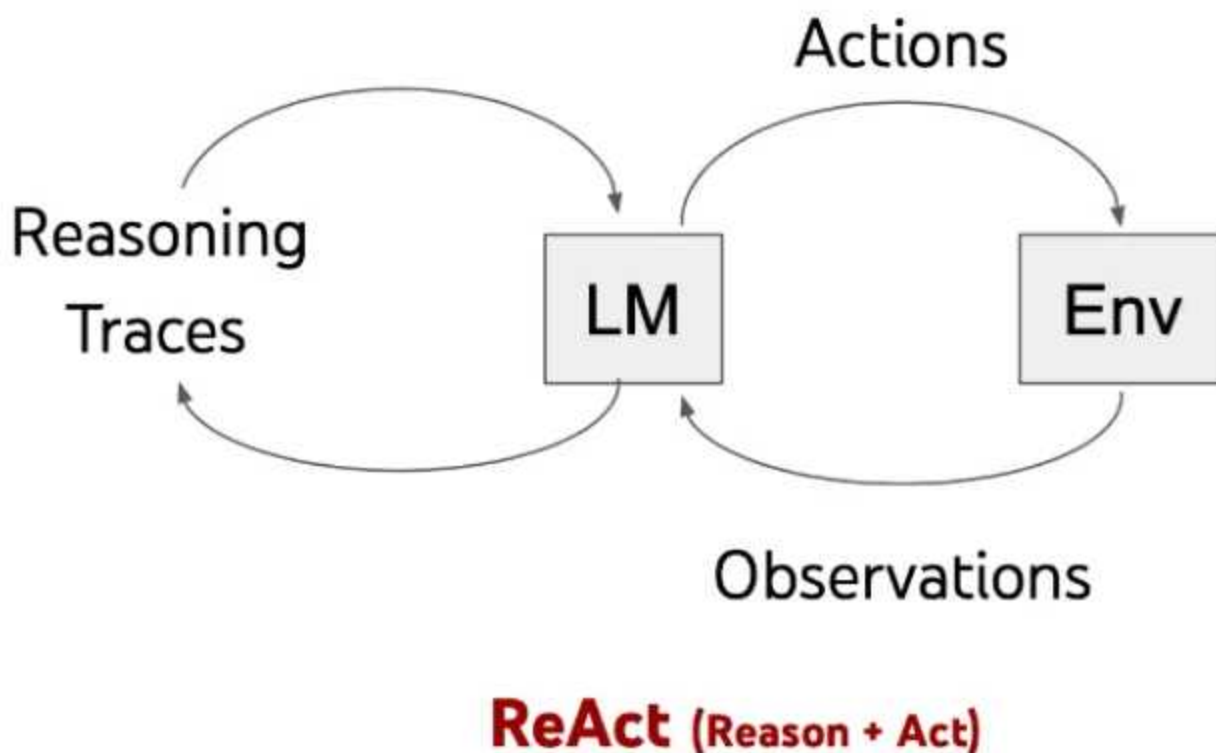


随着长上下文技术的进步，上下文工程已成为智能体和知识密集型推理领域的热门研究方法

1.1.5 智能体与ReAct架构

ReAct: AI Agent奠基工作

ReAct 是 Agent 领域一种通过“思考 – 行动 – 观察”循环（先生成推理思考文本明确目标与步骤，再执行工具调用等外部行动，最后基于行动反馈调整后续策略），让智能体在复杂任务中兼具推理能力与环境交互能力，从而提升决策可靠性的框架。



1.2 问题背景

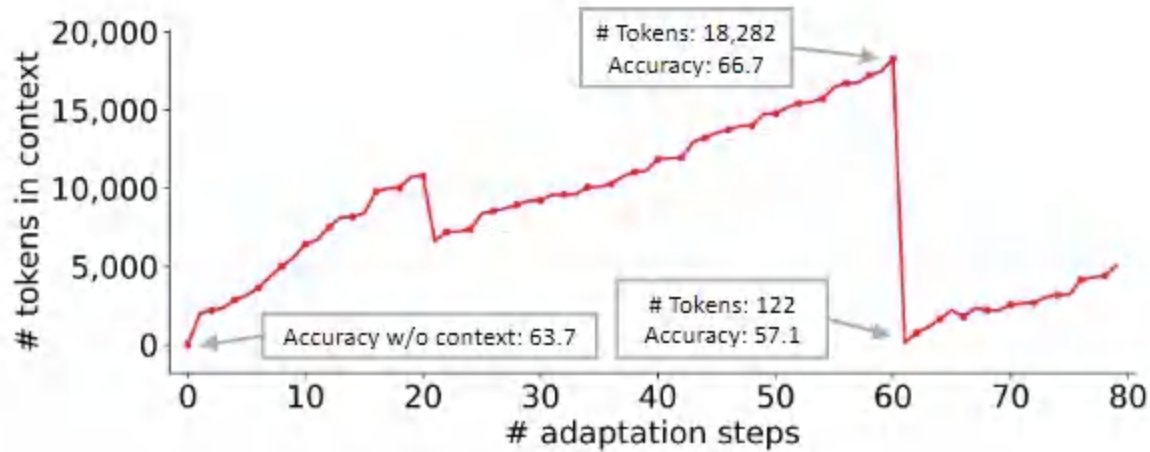
当前上下文工程研究工作遇到的挑战

1.2.1 简洁性偏差

- 核心问题：为追求指令简洁通用，主动删掉领域细节（如工具使用规则、常见错误处理）。
- 本质：错把“人类需要简洁”套用到 LLM，忽视其在长文本中更易提炼有效信息的特性。
- 影响：复杂任务（如金融推理、智能体交互）因缺乏关键策略，性能大幅下降。

1.2.2 上下文坍缩

- 核心问题：迭代重写上下文时，LLM 将长文本压缩为短而无意义的摘要，抹除历史积累的知识。
- 本质：无结构化更新机制，模型为降低处理难度，自主舍弃关键信息。
- 影响：知识积累断裂，前序任务学到的策略失效，性能断崖式下跌。



1.3 相关工作

1.3.1 ICL

in-context learning, 上下文学习

主要是few-shot技术，通过在提示中嵌入少量任务示例，让模型仅凭示例推断任务格式与目标输出，无需权重更新。

1.3.2 GEPA

基于**执行轨迹**的反射式**反馈迭代优化提示**，用**遗传帕累托搜索**保留高性能提示方案。

根据模型执行结果找问题，反复打磨提示并保留好用的版本。

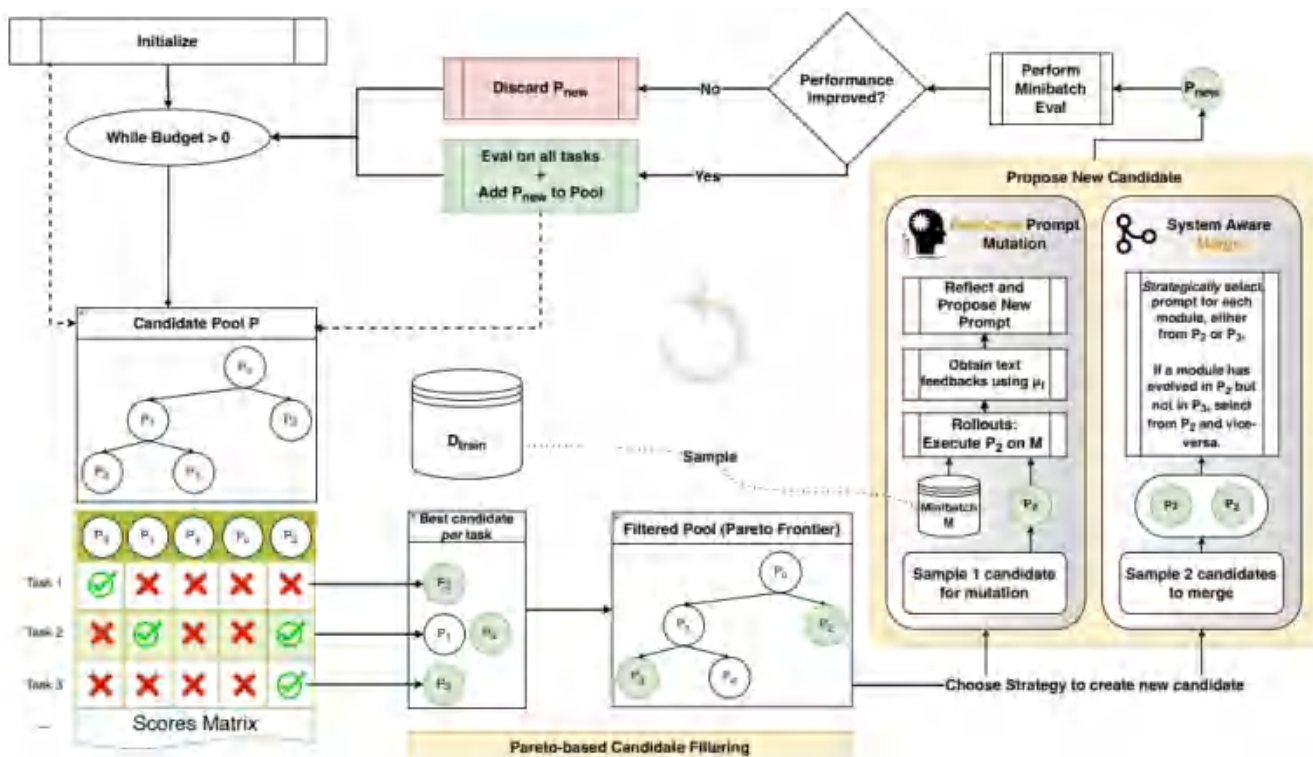


Figure 3: GEPA works iteratively—proposing a new candidate in every iteration by improving some existing candidates using one of the two strategies (Reflective Prompt Mutation (Section 3.2) or System Aware Merge (Appendix F)), first evaluating them on a minibatch, and if improved, evaluating on a larger dataset. Instead of selecting the best performing candidate to mutate always, which can lead to a local-optimum, GEPA introduces Pareto-based candidate sampling (Section 3.3), which filters and samples from the list of best candidates *per* task, ensuring sufficient diversity. Overall, these design decisions allow GEPA to be highly sample-efficient while demonstrating strong generalization.

1.3.3 Dynamic Cheatsheet

构建自适应外部记忆，积累过往任务的策略与代码片段，支持模型测试时复用知识。

给模型建个“错题本 + 技巧库”，攒着过往经验供后续复用。

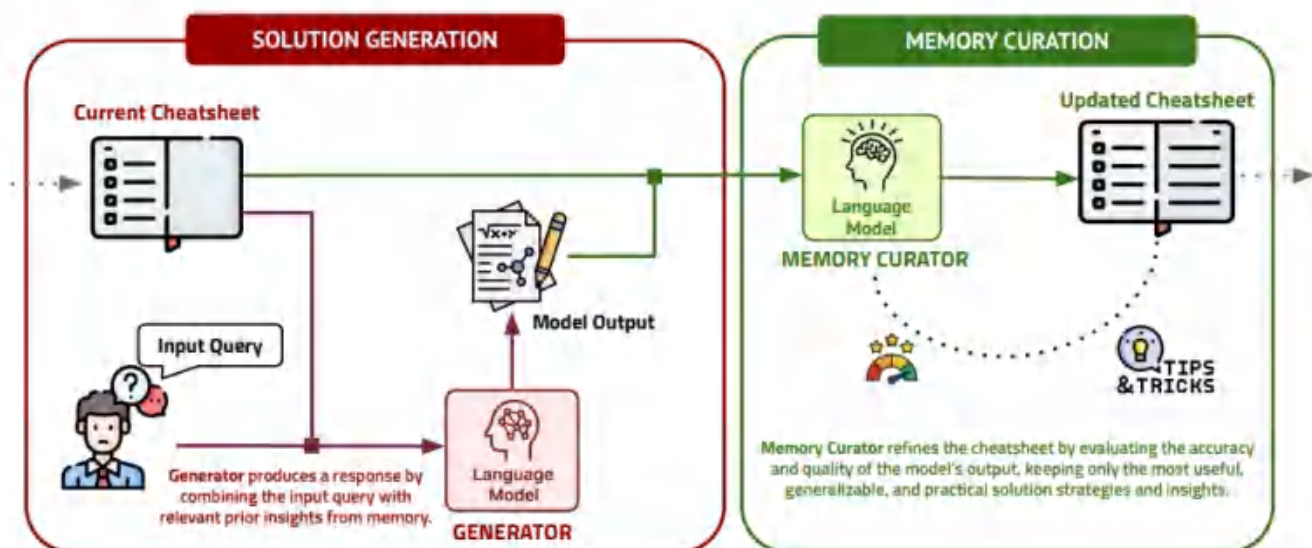


Figure 4: Illustration of *Dynamic Cheatsheet* (DC-Cu variant).

2 模型与方法

2.1 ACE框架

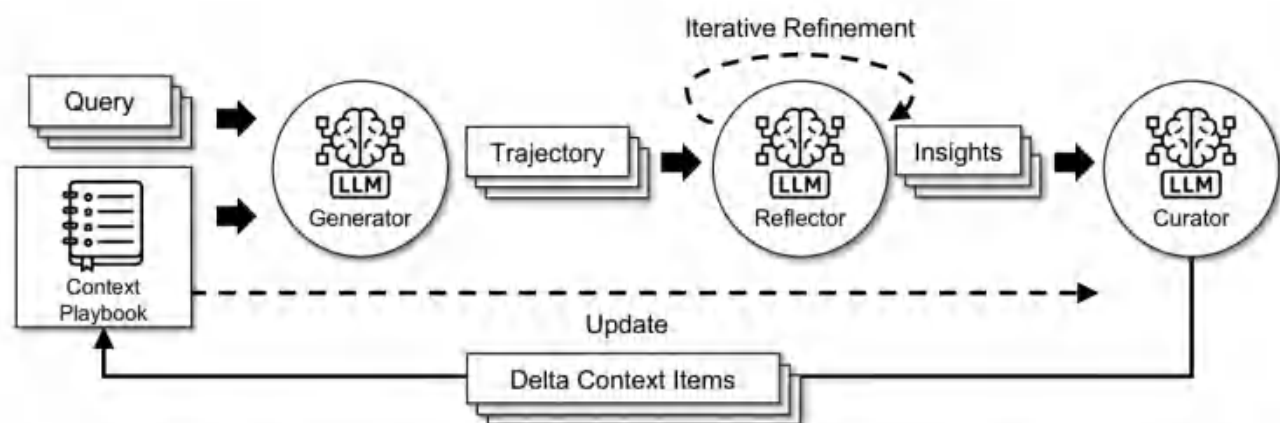


Figure 4: **The ACE Framework.** Inspired by Dynamic Cheatsheet, ACE adopts an agentic architecture with three specialized components: a Generator, a Reflector, and a Curator.

2.1.1 生成器（Generator）

- **核心职责：**聚焦“任务执行与轨迹生成”，是上下文信息的“源头产出者”。
- **核心功能：**针对新查询或任务，生成完整推理轨迹（如智能体 API 调用步骤、金融推理过程），过程中既呈现有效策略（如正确的 XBRL 实体识别逻辑），也暴露常见陷阱（如 API

分页错误)；同时可标记上下文中哪些条目有用或误导，为后续优化提供反馈。

2.1.2 反思器 (Reflector)

- **核心职责：**聚焦“经验提炼与错误诊断”，是上下文质量的“把关者”。
- **核心功能：**批判性分析生成器的推理轨迹，从成功案例中总结可复用策略（如“账单拆分需通过 Phone app 识别室友”），从失败中定位根因（如“数值计算错误源于未验证 XBRL 单位”）；支持多轮迭代精炼，必要时可调用外部工具验证（如医疗领域的证据核查），确保提炼的洞见准确、具体。

2.1.3 整理器 (Curator)

- **核心职责：**聚焦“信息整合与结构化管理”，是上下文演进的“组织者”。
- **核心功能：**将反思器提炼的经验合成为紧凑的“增量条目”（含唯一 ID、使用频次等元数据），通过轻量级非 LLM 逻辑（如语义嵌入去重），确定性合并到现有上下文中；支持局部化更新（仅改相关条目）与并行合并，避免整体重写导致的信息丢失，同时通过“grow-and-refine”机制平衡上下文扩展与冗余控制。

2.2 实现细节

2.2.1 ContextPlaybook

The Generated ACE Playbook on AppWorld

STRATEGIES AND HARD RULES

[shr-00009]

When processing time-sensitive transactions involving specific relationships: always resolve identities from the correct source app (phone contacts), use proper datetime range comparisons instead of string matching, and verify all filtering criteria (relationship + time) are met before processing items. This ensures accurate identification and processing of the right transactions.

USEFUL CODE SNIPPETS AND TEMPLATES

[code-00013]

For efficient artist aggregation when processing songs, use defaultdict(list) to map song titles to artist names:

```
from collections import defaultdict artist_map = defaultdict(list) for song in songs:
    artist_map[song['title']].extend([artist['name'] for artist in song['artists']])
```

TROUBLESHOOTING AND PITFALLS

[ts-00003]

If authentication fails, troubleshoot systematically: try phone number instead of email as username, clean credentials from supervisor, check API documentation for correct parameters etc. Do not proceed with workarounds.

2.2.2 增量式差异更新

- **核心载体**：将上下文拆分为**结构化“bullet 条目”**，每个条目含两部分 —— 元数据（唯一 ID、“有用 / 有害”计数器）、内容（可复用策略、领域概念、失败模式等小粒度单元）。
- **更新逻辑**：不重写整个上下文，仅生成“delta 上下文”（反思器提炼的少量候选条目），由整理器通过**轻量级非 LLM 逻辑**确定性合并：仅更新相关条目，新增条目附加新 ID，现有条目原位调整（如递增计数器）。
- **关键特性**：支持本地化更新（聚焦相关条目）、并行合并（多 delta 可同步整合），避免全量重写导致的信息丢失，同时降低计算 latency。

（与Dynamic Cheatsheet做对比，DC让LLM对全上下文进行分析补充，成本高）

2.2.3 冗余控制

- **核心机制**：“grow-and-refine”（增长 – 优化）双阶段策略，平衡上下文扩展与冗余。
- **去重实现**：通过**语义嵌入对比**识别重复条目，自动 pruning 冗余内容 —— 不依赖 LLM 重写，仅通过**向量相似度判断**条目语义重合度。
- **触发时机**：支持两种模式 —— 主动式（每轮 delta 更新后执行）、惰性式（仅当上下文超出

模型窗口时执行），适配不同 latency-accuracy 需求。

(文本查重去冗余)

2.2.4 离线预热

- **核心目标**：为在线适应提供**高质量初始上下文**，避免从零开始积累策略。
- **实现流程**：先在训练集上完成离线适配 —— 利用训练数据（含 GT labels），通过生成器 – 反思器 – 整理器闭环，构建初始策略库（如金融领域的 XBRL 规则、智能体的 API 调用规范）。
- **衔接逻辑**：在线适应阶段直接复用该初始库，仅针对新场景 / 错误生成增量更新，无需重复优化通用策略。实验验证（消融实验）：离线预热使 AppWorld 在线适应平均准确率再提升 3.4%，挑战集 TGC 从 61.4% 升至 66.0%。

(冷启动) （对于金融分析领域提示词，复用金融文档手册）

3 数据与实验

3.1 基准实验

3.1.1 AppWorld Agent Benchmark

智能体任务

Method	GT Labels	Test-Normal		Test-Challenge		Average
		TGC↑	SGC↑	TGC↑	SGC↑	
DeepSeek-V3.1 as Base LLM						
ReAct		63.7	42.9	41.5	21.6	42.4
Offline Adaptation						
ReAct + ICL	✓	64.3 _{+0.6}	46.4 _{+3.5}	46.0 _{+4.5}	27.3 _{+5.7}	46.0 _{+3.6}
ReAct + GEPA	✓	64.9 _{+1.2}	44.6 _{+1.7}	46.0 _{+4.5}	30.2 _{+8.6}	46.4 _{+4.0}
ReAct + ACE	✓	76.2 _{+12.5}	64.3 _{+21.4}	57.3 _{+15.8}	39.6 _{+18.0}	59.4 _{+17.0}
ReAct + ACE	✗	75.0 _{+11.3}	64.3 _{+21.4}	54.4 _{+12.9}	35.2 _{+13.6}	57.2 _{+14.8}
Online Adaptation						
ReAct + DC (CU)	✗	65.5 _{+1.8}	58.9 _{+16.0}	52.3 _{+10.8}	30.8 _{+9.2}	51.9 _{+9.5}
ReAct + ACE	✗	69.6 _{+5.9}	53.6 _{+10.7}	66.0 _{+24.5}	48.9 _{+27.3}	59.5 _{+17.1}

Table 1: **Results on the AppWorld Agent Benchmark.** "GT labels" indicates whether ground-truth labels are available to the Reflector during adaptation. We evaluate the ACE framework against multiple baselines on top of the official ReAct implementation, both for offline and online context adaptation. ReAct + ACE outperforms selected baselines by an average of 10.6%, and could achieve good performance even without access to GT labels.

指标：

TGC (Task Goal Completion, 任务目标完成率)：关注原子任务

SGC (Scenario Goal Completion, 场景目标完成率)：贴合实际应用

GT Labels：是否为反思器提供对错反馈 (ground-true)

3.1.2 Financial Analysis Benchmark

专业领域任务(金融分析)

Method	GT Labels	FINER (Acc $\uparrow$)	Formula (Acc $\uparrow$)	Average
DeepSeek-V3.1 as Base LLM				
Base LLM		70.7	67.5	69.1
Offline Adaptation				
ICL	✓	72.3 _{+1.6}	67.0 _{-0.5}	69.6 _{+0.5}
MIPROv2	✓	72.4 _{+1.7}	69.5 _{+2.0}	70.9 _{+1.8}
GEPA	✓	73.5 _{+2.8}	71.5 _{+4.0}	72.5 _{+3.4}
ACE	✓	78.3 _{+7.6}	85.5 _{+18.0}	81.9 _{+12.8}
ACE	✗	71.1 _{+0.4}	83.0 _{+13.5}	77.1 _{+8.0}
Online Adaptation				
DC (CU)	✓	74.2 _{+3.5}	69.5 _{+2.0}	71.8 _{+2.7}
DC (CU)	✗	68.3 _{-2.4}	62.5 _{-5.0}	65.4 _{-3.7}
ACE	✓	76.7 _{+6.0}	76.5 _{+9.0}	76.6 _{+7.5}
ACE	✗	67.3 _{-3.4}	78.5 _{+11.0}	72.9 _{+3.8}

Table 2: **Results on Financial Analysis Benchmark.** "GT labels" indicates whether ground-truth labels are available to the Reflector during adaptation. With GT labels, ACE outperforms selected baselines by an average of 8.6%, highlighting the advantage of structured and evolving contexts for domain-specific reasoning. However, we also observe that in the absence of reliable feedback signals (e.g., ground-truth labels or execution outcomes), both ACE and other adaptive methods such as Dynamic Cheatsheet may degrade, suggesting that context adaptation depends critically on feedback quality.

3.2 消融实验

离线/在线适配，反思器，多轮适应，离线预热

Method	GT Labels	Test-Normal		Test-Challenge		Average
		TGC↑	SGC↑	TGC↑	SGC↑	
DeepSeek-V3.1 as Base LLM						
ReAct		63.7	42.9	41.5	21.6	42.4
Offline Adaptation						
ReAct + ACE w/o Reflector or multi-epoch	✓	70.8 _{+7.1}	55.4 _{+12.5}	55.9 _{+14.4}	38.1 _{-17.5}	55.1 _{+12.7}
ReAct + ACE w/o multi-epoch	✓	72.0 _{+8.3}	60.7 _{+17.8}	54.9 _{+13.4}	39.6 _{-18.0}	56.8 _{+14.4}
ReAct + ACE	✓	76.2 _{+12.5}	64.3 _{+21.4}	57.3 _{+15.8}	39.6 _{-18.0}	59.4 _{+17.0}
Online Adaptation						
ReAct + ACE	✗	67.9 _{+4.2}	51.8 _{+8.9}	61.4 _{+19.0}	43.2 _{-21.6}	56.1 _{+13.7}
ReAct + ACE + offline warmup	✗	69.6 _{+5.9}	53.6 _{+10.7}	66.0 _{+24.5}	48.9 _{-27.3}	59.5 _{+17.1}

Table 3: **Ablation Studies on AppWorld.** We study how particular design choices of ACE (iterative refinement, multi-epoch adaptation, and offline warmup) could help high-quality context adaptation.

3.3 性能与成本对比

Method	Latency (s)↓	# Rollouts↓
ReAct + GEPA	53898	1434
ReAct + ACE	9517 _(+82.3%)	357 _(-75.1%)

(a) Offline (AppWorld).

Method	Latency (s)↓	Token Cost (\$)↓
DC (CU)	65104	17.7
ACE	5503 _(-91.5%)	2.9 _(-83.6%)

(b) Online (FiNER).

Table 4: **Cost and Speed Analysis.** We measure the context adaptation latency, number of rollouts, and dollar costs of ACE against GEPA (offline) and DC (online).

4 总结与展望

4.1 创新点

1. 提出“**演进式上下文**”范式：突破传统“简洁指令”认知，将上下文设计为可积累、可精炼的“操作手册”，适配 LLM 在长文本中自主提炼相关性的特性。
2. 三模块分工架构：通过生成器（产轨迹）、反思器（提洞见）、整理器（做管理）的**模块化设计**，分离信息生成、提炼与整合，避免单一模块过载。
3. 增量更新与增长 – 优化机制：以结构化条目实现局部**增量更新**，替代整体重写，解决上下文坍塌；通过语义去重平衡扩展与冗余，规避简洁性偏差。
4. 双场景适配与无监督能力：同时**支持离线（系统提示优化）与在线（实时记忆更新）场景**，仅依赖执行反馈即可实现有效适配，无需大量标注数据。

4.2 局限性

1. 依赖高质量反馈信号：缺乏可靠执行反馈（如**无标注**、无 API 结果）时，上下文易引入噪声，**性能显著下降**（如 FiNER 无 GT labels 时仅较基线 + 0.4%）。
2. 模块性能强关联：**反思器**的洞见提取能力直接决定上下文质量，若模型无法识别关键错误或有效策略，会影响整体效果。
3. **长上下文成本未充分验证**：虽提及 KV 缓存复用，但未量化超长篇上下文（如 10 万 tokens 以上）的推理 latency 与内存占用，可扩展性存疑。
4. 适用场景有局限：对需**简洁指令的任务（如简单问答、固定规则游戏）增益有限**，更适配复杂领域推理或智能体交互场景。

4.3 成果公开

个人思考

- 从ICL到RAG到ACE（“多快好省”中舍弃“快”和“省”，追求“多”和“好”）
- ACE是否适用于ACSL生成：有反馈信息和目标（有构建高质量**反思器**的潜力）